ISEL
DEETC
Departamento de Engenharia Electrónica e de Telecomunicações e de Computadores

Aula 6

Sumário

- Circuitos combinatórios

Circuitos digitais

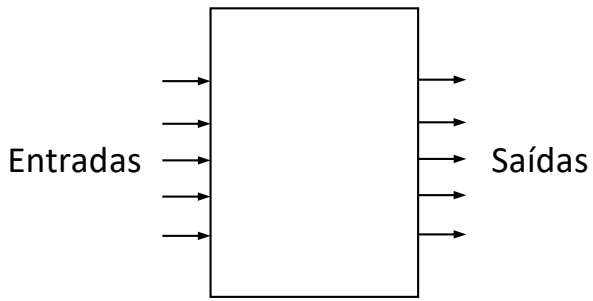
- Os circuitos digitais podem ser categorizados como:
  - Circuitos combinatórios
    - O valor da saída num determinado momento depende exclusivamente do valor das entradas nesse momento
  - Circuitos sequenciais
    - O valor da saída num determinado momento depende da sequência temporal do valor das entradas até esse momento

Circuitos combinatórios

- Um circuito combinatório é definido através da função lógica que representa cada uma das suas saídas
- É implementado utilizando portas lógicas que realizam as funções dos diversos operadores booleanos

Circuitos combinatórios

- Um circuito combinatório é definido através da função lógica que representa cada uma das suas saídas
- É implementado utilizando portas lógicas que realizam as funções dos diversos operadores booleanos

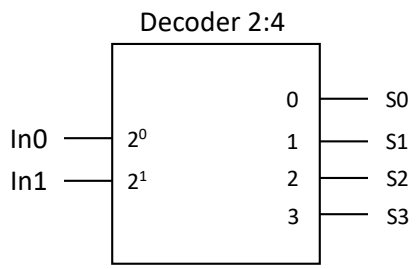


Circuitos combinatórios típicos

- Exemplos de módulos frequentemente usados como base para circuitos com maior complexidade:
  - *Decoder* (descodificador)
  - *Encoder* (codificador)
  - *Multiplexer* (multiplexador)
  - *Demultiplexer* (desmultiplexador)

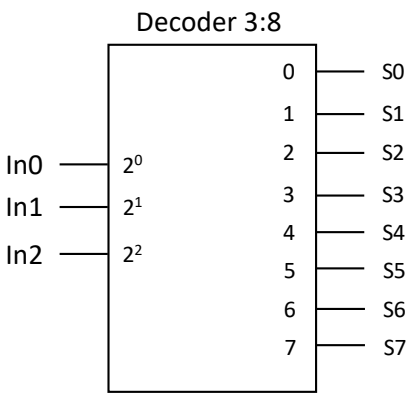
Decoder

- Converte *n* bits de entrada em *m* bits de saída ($n \leq m \leq 2^n$)
  - Tipicamente, converte *n* bits de entrada em 2^n bits de saída
- Cada código válido de entrada resulta numa única saída ativa
- Gera todos os mintermos dos bits de entrada
  - Exemplo:
    - Decoder de 2:4



Decoder 3-bit (3:8)

- Símbolo e tabela de verdade



IN2	IN1	IN0	S0	S1	S2	S3	S4	S5	S6	S7
0	0	0	1	0	0	0	0	0	0	0
0	0	1	0	1	0	0	0	0	0	0
0	1	0	0	0	1	0	0	0	0	0
0	1	1	0	0	0	1	0	0	0	0
1	0	0	0	0	0	0	1	0	0	0
1	0	1	0	0	0	0	0	1	0	0
1	1	0	0	0	0	0	0	0	1	0
1	1	1	0	0	0	0	0	0	0	1

Decoder 3-bit (3:8) Funções lógicas

- Funções das saídas

IN2	IN1	IN0	S0	S1	S2	S3	S4	S5	S6	S7
0	0	0	1	0	0	0	0	0	0	0
0	0	1	0	1	0	0	0	0	0	0
0	1	0	0	0	1	0	0	0	0	0
0	1	1	0	0	0	1	0	0	0	0
1	0	0	0	0	0	0	1	0	0	0
1	0	1	0	0	0	0	0	1	0	0
1	1	0	0	0	0	0	0	0	1	0
1	1	1	0	0	0	0	0	0	0	1

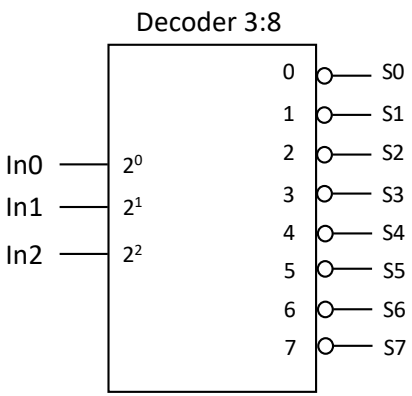
$$S0 = \overline{IN2} \cdot \overline{IN1} \cdot \overline{IN0}$$
$$S1 = \overline{IN2} \cdot \overline{IN1} \cdot IN0$$
$$S2 = \overline{IN2} \cdot IN1 \cdot \overline{IN0}$$
$$S3 = \overline{IN2} \cdot IN1 \cdot IN0$$
$$S4 = IN2 \cdot \overline{IN1} \cdot \overline{IN0}$$
$$S5 = IN2 \cdot \overline{IN1} \cdot IN0$$
$$S6 = IN2 \cdot IN1 \cdot \overline{IN0}$$
$$S7 = IN2 \cdot IN1 \cdot IN0$$

Circuitos digitais
níveis lógicos - *active_high* e *active_low*

- Os circuitos digitais utilizam grandezas físicas (e.g. tensão, corrente) para representar (codificar) os valores lógicos binários da álgebra de Boole
  - Cada tecnologia tem o seu próprio mapeamento
- Um dos mapeamentos tipicamente utilizados, consiste em fazer corresponder os valores lógicos binários a intervalos de tensão
  - Valor de tensão HIGH – [2; 5] V
  - Valor de tensão LOW – [0; 0,8] V
- Adicionalmente, para cada entrada/saída há que definir quando é que a mesma se considera ativa:
  - Valor de tensão HIGH – caso em que se denomina a entrada/saída como *active_high*
  - Valor de tensão LOW – caso em que se denomina a entrada/saída como *active_low*
- Para simplificar, consideraremos que:
  - o valor lógico numérico 1 está associado ao valor de tensão HIGH (1 <–> HIGH)
  - o valor lógico numérico 0 está associado ao valor de tensão LOW (0 <–> LOW)

Decoder 3-bit (3:8) com saídas *active_low*

- Símbolo e tabela de verdade



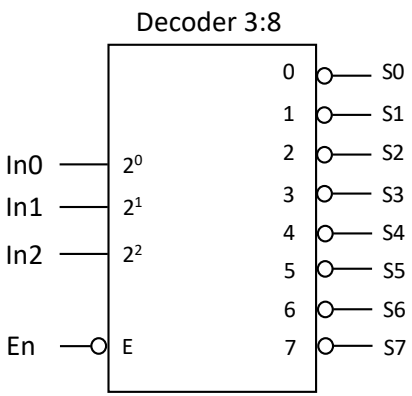
IN2	IN1	IN0	S0	S1	S2	S3	S4	S5	S6	S7
0	0	0	0	1	1	1	1	1	1	1
0	0	1	1	0	1	1	1	1	1	1
0	1	0	1	1	0	1	1	1	1	1
0	1	1	1	1	1	0	1	1	1	1
1	0	0	1	1	1	1	0	1	1	1
1	0	1	1	1	1	1	1	0	1	1
1	1	0	1	1	1	1	1	1	0	1
1	1	1	1	1	1	1	1	1	1	0

Entrada de *Enable*

- A entrada de *Enable* permite controlar quando o dispositivo executa a sua função normal
- Quando desactivada, o circuito apresenta na sua saída sempre o mesmo valor (independentemente do valor das outras entradas)
- O valor lógico das saídas, quando a entrada *Enable* está desactivada, é determinado pelo fabricante (normalmente corresponde ao valor lógico 0 caso a saída seja *active_high*, ou ao valor lógico 1 caso a saída seja *active_low)*
- Exemplo:
  - *Decoder* com *enable*

Decoder 3-bit (3:8) com saídas *active_low* e *enable* (*active_low*)

- Símbolo e tabela de verdade



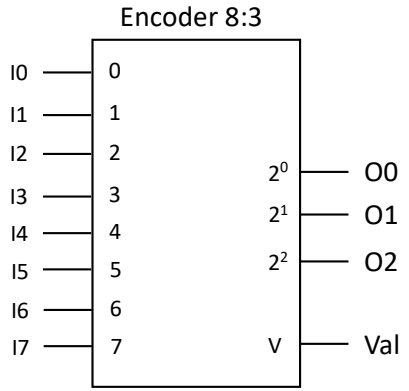
EN	IN2	IN1	IN0	S0	S1	S2	S3	S4	S5	S6	S7
1	-	-	-	1	1	1	1	1	1	1	1
0	0	0	0	0	1	1	1	1	1	1	1
0	0	0	1	1	0	1	1	1	1	1	1
0	0	1	0	1	1	0	1	1	1	1	1
0	0	1	1	1	1	1	0	1	1	1	1
0	1	0	0	1	1	1	1	0	1	1	1
0	1	0	1	1	1	1	1	1	0	1	1
0	1	1	0	1	1	1	1	1	1	0	1
0	1	1	1	1	1	1	1	1	1	1	0

Encoder

- Converte *m* bits de entrada em *n* bits de saída ($n \leq m \leq 2^n$)
  - Tipicamente, converte 2^n bits de entrada em *n* bits de saída
  - *n* representa o número de bits do palavra de código na saída
  - *m* representa o número de entradas distintas passíveis de serem codificadas
- Apenas considera uma entrada ativa a cada momento
- É a função inversa do *Decoder*

Encoder 8:3

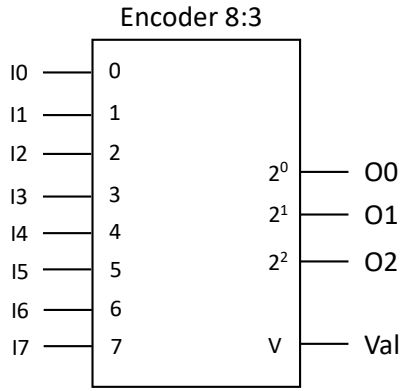
- Símbolo e tabela de verdade



I0	I1	I2	I3	I4	I5	I6	I7	O2	O1	O0	Val
0	0	0	0	0	0	0	0	-	-	-	0
1	0	0	0	0	0	0	0	0	0	0	1
0	1	0	0	0	0	0	0	0	0	1	1
0	0	1	0	0	0	0	0	0	1	0	1
0	0	0	1	0	0	0	0	0	1	1	1
0	0	0	0	1	0	0	0	1	0	0	1
0	0	0	0	0	1	0	0	1	0	1	1
0	0	0	0	0	0	1	0	1	1	0	1
0	0	0	0	0	0	0	1	1	1	1	1

Encoder 8:3

- Símbolo e tabela de verdade

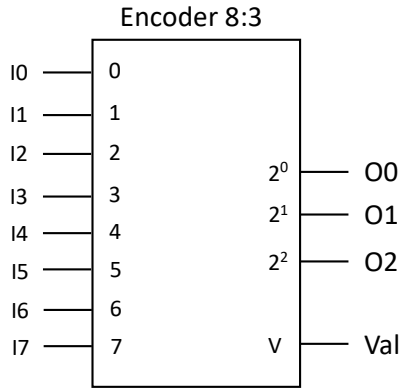


I0	I1	I2	I3	I4	I5	I6	I7	O2	O1	O0	Val
0	0	0	0	0	0	0	0	-	-	-	0
1	0	0	0	0	0	0	0	0	0	0	1
0	1	0	0	0	0	0	0	0	0	1	1
0	0	1	0	0	0	0	0	0	1	0	1
0	0	0	1	0	0	0	0	0	1	1	1
0	0	0	0	1	0	0	0	1	0	0	1
0	0	0	0	0	1	0	0	1	0	1	1
0	0	0	0	0	0	1	0	1	1	0	1
0	0	0	0	0	0	0	1	1	1	1	1

Tabela de verdade não está completa!!!
Faltam 247 combinações!!!

Encoder 8:3

- Símbolo e tabela de verdade



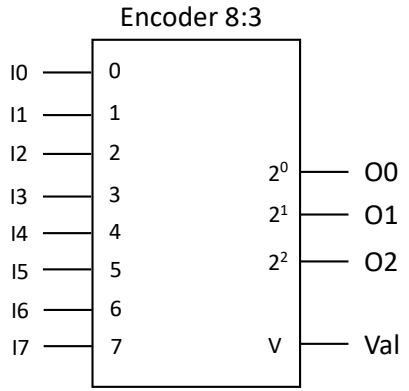
I0	I1	I2	I3	I4	I5	I6	I7	O2	O1	O0	Val
0	0	0	0	0	0	0	0	-	-	-	0
1	0	0	0	0	0	0	0	0	0	0	1
0	1	0	0	0	0	0	0	0	0	1	1
0	0	1	0	0	0	0	0	0	1	0	1
0	0	0	1	0	0	0	0	0	1	1	1
0	0	0	0	1	0	0	0	1	0	0	1
0	0	0	0	0	1	0	0	1	0	1	1
0	0	0	0	0	0	1	0	1	1	0	1
0	0	0	0	0	0	0	1	1	1	1	1
(restantes)								-	-	-	?

Priority encoder

- Converte *m* bits de entrada em *n* bits de saída ($n \leq m \leq 2^n$)
  - Tipicamente, converte 2^n bits de entrada em *n* bits de saída
  - *n* representa o número de bits do palavra de código na saída
  - *m* representa o número de entradas distintas passíveis de serem codificadas
- Permite várias entradas ativas simultaneamente, mas apenas o código da entrada com maior prioridade é disponibilizado na saída

Priority encoder 8:3

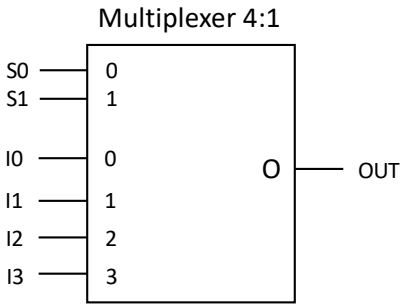
- Símbolo e tabela de verdade



I0	I1	I2	I3	I4	I5	I6	I7	O2	O1	O0	Val
0	0	0	0	0	0	0	0	-	-	-	0
1	-	-	-	-	-	-	-	0	0	0	1
0	1	-	-	-	-	-	-	0	0	1	1
0	0	1	-	-	-	-	-	0	1	0	1
0	0	0	1	-	-	-	-	0	1	1	1
0	0	0	0	1	-	-	-	1	0	0	1
0	0	0	0	0	1	-	-	1	0	1	1
0	0	0	0	0	0	1	-	1	1	0	1
0	0	0	0	0	0	0	1	1	1	1	1

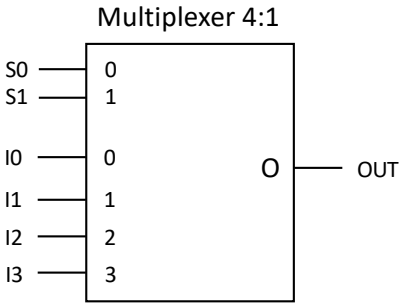
Multiplexer

- Seleciona uma de *n* entradas, em exclusão, para a saída
- A seleção é feita através de uma entrada de seleção com $\lceil \log_2 n \rceil$ bits que codifica a entrada a selecionar
- A implementação é feita à custa da descodificação da entrada de selecção
- Exemplo:

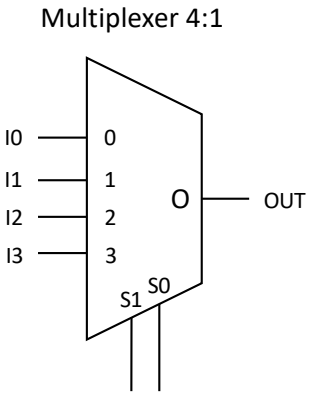


Multiplexer 4:1

- Símbolo e tabela de verdade



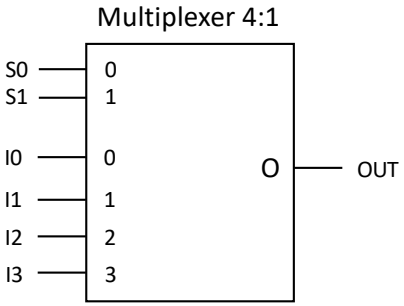
≡



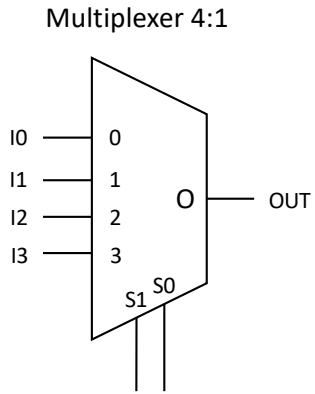
S1	S0	OUT
0	0	I0
0	1	I1
1	0	I2
1	1	I3

Multiplexer 4:1

- Símbolo e tabela de verdade



≡

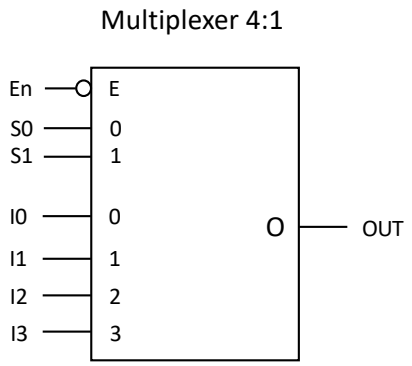


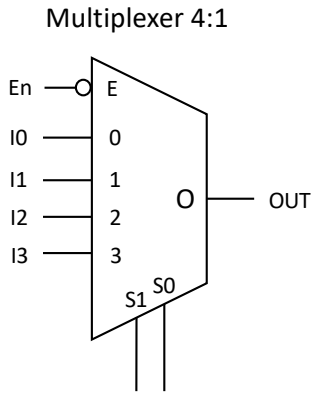
S1	S0	OUT
0	0	I0
0	1	I1
1	0	I2
1	1	I3

$$\mathrm{OUT} = \overline{\mathrm{S1}}\,\overline{\mathrm{S0}}\,\mathrm{I0} + \overline{\mathrm{S1}}\,\mathrm{S0}\,\mathrm{I1} + \mathrm{S1}\,\overline{\mathrm{S0}}\,\mathrm{I2} + \mathrm{S1}\,\mathrm{S0}\,\mathrm{I3}$$

Multiplexer 4:1 com *Enable active_low*

- Símbolo e tabela de verdade

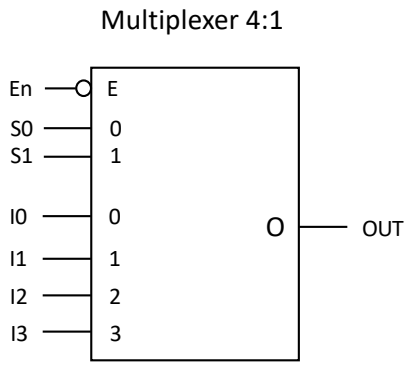


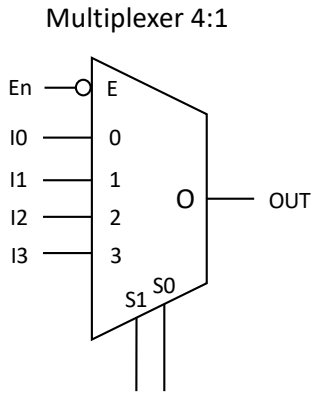


E	S1	S0	OUT
1	-	-	0
0	0	0	I0
0	0	1	I1
0	1	0	I2
0	1	1	I3

Multiplexer 4:1 com *Enable active_low*

- Símbolo e tabela de verdade



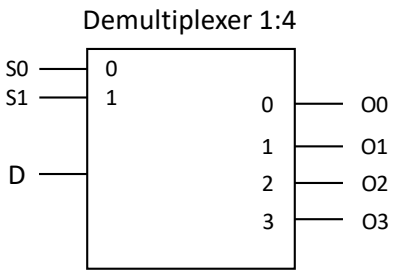


E	S1	S0	OUT
1	-	-	0
0	0	0	I0
0	0	1	I1
0	1	0	I2
0	1	1	I3

$$\mathrm{OUT} = \overline{\mathrm{EN}}\,(\overline{\mathrm{S1}}\,\overline{\mathrm{S0}}\,\mathrm{I0} + \overline{\mathrm{S1}}\,\mathrm{S0}\,\mathrm{I1} + \mathrm{S1}\,\overline{\mathrm{S0}}\,\mathrm{I2} + \mathrm{S1}\,\mathrm{S0}\,\mathrm{I3})$$

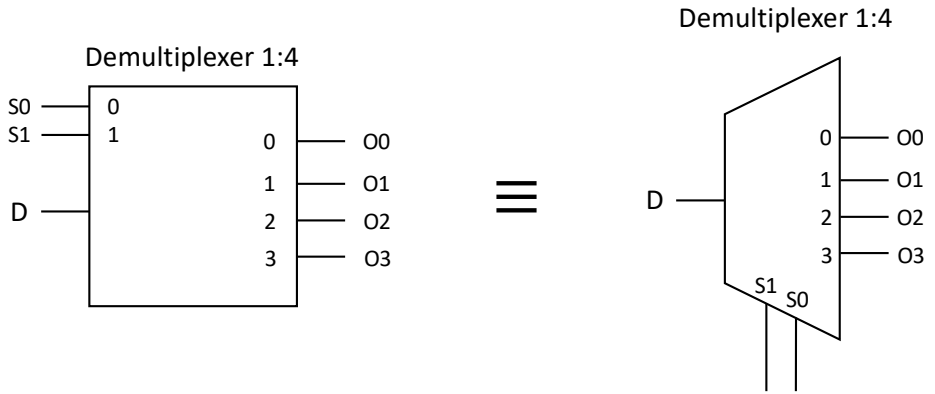
Demultiplexer

- Seleciona para qual das *n* saídas deve ser encaminhado o valor da entrada
- A selecção é feita através de uma entrada de selecção com $\lceil \log_2 n \rceil$ bits que codifica a saída a seleccionar
- A implementação é feita à custa da descodificação da entrada de selecção
- Exemplo:



Demultiplexer 1:4

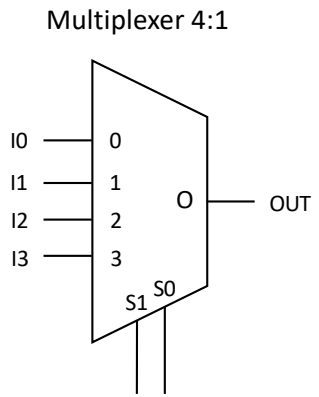
- Símbolo e tabela de verdade



S1	S0	O3	O2	O1	O0
0	0	0	0	0	D
0	1	0	0	D	0
1	0	0	D	0	0
1	1	D	0	0	0

Implementação de funções com *multiplexers*

- Um *multiplexer* pode ser utilizado para implementar funções lógicas
- Internamente, o *multiplexer* gera uma soma de produtos das variáveis de seleção
  - Exemplo: *multiplexer* 4:1

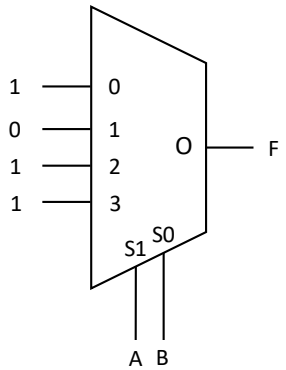


$$\mathrm{OUT} = \overline{\mathrm{S1}}\,\overline{\mathrm{S0}}\,\mathrm{I0} + \overline{\mathrm{S1}}\,\mathrm{S0}\,\mathrm{I1} + \mathrm{S1}\,\overline{\mathrm{S0}}\,\mathrm{I2} + \mathrm{S1}\,\mathrm{S0}\,\mathrm{I3}$$

Implementação de funções com *multiplexers*

- Um *multiplexer* 4:1 pode ser utilizado para implementar qualquer função lógica de 2 variáveis
- Consideremos a função descrita na tabela de verdade abaixo e a sua implementação com recurso a um *multiplexer* 4:1

A	B	F
0	0	1
0	1	0
1	0	1
1	1	1



$$F = \overline{S1}\,\overline{S0}\,I0 + \overline{S1}\,S0\,I1 + S1\,\overline{S0}\,I2 + S1\,S0\,I3$$

$$F = \overline{A}\,\overline{B}\,1 + \overline{A}\,B\,0 + A\,\overline{B}\,1 + A\,B\,1$$

$$F = \overline{A}\,\overline{B} + A\,\overline{B} + A\,B$$

Temas abordados

Sumário

- Circuitos combinatórios

Próxima aula

- Laboratório A